Using the AIX Lab-Atlas skills

in Claude Design

For	Designers working in Claude Design. Setup takes about 10 minutes, once per project.
Source	https://github.com/RPalis/AIXLab-Atlas. This is a team guide, not an official Anthropic document.
Status	Tested in one Claude Design session: connecting the repository, copying the skills, adding the rule file and running `/help atlas`. Other commands have not been tested yet.

What you are setting up

The repository holds 30 skills. Each skill is a folder with a `SKILL.md` file of instructions, for example `audit`, `brief` and `persona`. You copy those folders into your Claude Design project, add one rule file, and then ask for a skill by name in chat. Nothing is installed on your computer and you do not need a terminal.

Step 1. Connect the repository

1. Open a project in Claude Design.
2. In chat, send: `Connect the GitHub repo RPalis/AIXLab-Atlas, branch main.`
3. If a repository picker appears, choose `RPalis/AIXLab-Atlas`.
4. If the assistant says it cannot see the repository, install the GitHub App when prompted. This happens with private repositories.

> **CHECK** The assistant can list the repository's `skills/` folder.

Step 2. Copy the skills into the project

1. If this project already holds a `skills/` folder from an earlier copy, delete it first. An old copy is the only way to end up with retired or duplicate skills.
2. Send: `Copy the whole skills/ folder from RPalis/AIXLab-Atlas into this project, keeping the folder structure.`
3. Ask: `List the project's top-level folders. Every skill must sit at skills/<name>/SKILL.md.`
4. If the folders landed at the project root, send: `Move all of them under skills/.` This happened the first time.

> **CHECK** `skills/help/SKILL.md` exists, and `skills/` holds exactly 30 folders. The repository's `install.sh` is for terminal installs and is not needed here.

Step 3. Add the command rule

This file is what makes `/audit` and the other commands work. Send: `Create CLAUDE.md at the project root with exactly this text.` Then paste the block below. The project applies `CLAUDE.md` to every conversation in it. Repeat this step in each new project.

```
# Skill commands
The folder `skills/` holds the AIX Lab-Atlas skill suite (copied from RPalis/AIXLab-Atla
s).
When a user message starts with `/<name>` (for example `/help`, `/showcase`, `/audit craft
`),
treat it as a command to run that skill:
1. Read `skills/<name>/SKILL.md`. Read its `references/` and `templates/` files as the
   skill instructs.
2. Follow it exactly, using any word after the name as the mode or argument (`/audit a11y`
   means audit in a11y mode; `/brief meal plans for housemates` passes the text as the ide
a).
3. Do not answer that `/` commands are unsupported.
4. `/help` with no argument runs help's default mode. `/help atlas` runs atlas mode.
5. If `skills/<name>/` does not exist, list the folder names in `skills/` and ask which
   one was meant.
6. Each skill has an `## Environment` section. If it needs something this project does not
   provide, such as a shell or Node.js, say so, return the output in the conversation, and
   never claim a file was written or a command run when it was not.
The skill's `docs/...` paths mean this project's docs folder. The skill's own Output
Discipline applies: short, bullet-first chat replies.
```

Step 4. Invoke a skill

Type a slash, the skill name, and an optional mode or idea, then send. If you do not know where to start, send `/help`. It reads your project's files and recommends the next one to three skills, each tied to a file it found or did not find.

```
/help
/help atlas
/brief an app to share weekly meal plans with housemates
/audit craft
```

The slash is optional. "Run the audit skill in craft mode" does the same thing.

The commands

Phase	Commands (modes in brackets)
Router	help [default, atlas]. See the note in step 3 about `/help`.
Before Discover	`/setup`, `/brief`
Discover	`/workshop` [plan, ingest], `/competitor-analysis` [discover, add, analyse], `/ux-benchmark`, `/interview`, `/interview-analysis`, `/persona` [card, day], `/journey` [map, blueprint, pain-points, solutions, experience], `/problem-statement`, `/metrics`
Decide	`/brainstorming`, `/information-architecture`, `/map` [objects, site, flow, audit, document], `/interrogate`, `/decisions`
Design	`/prototype`, `/styleguide` [default, forge], `/audit` [craft, heuristic, code, spec, a11y], `/refine`, `/storybook`, `/figma` [capture, import, sync]
Deliver	`/usability-test-plan`, `/package` [full, lite], `/showcase`, `/guide`, `/requirements`

Phase	Commands (modes in brackets)
Any time	`/wiki` [compile, note], `/skill-manager`

What each skill needs

Every skill states what it needs in its own `## Environment` section. In short:

Needs	Skills
The conversation only (saving a file is optional)	brainstorming, interrogate, problem-statement, usability-test-plan, information-architecture, interview, interview-analysis, persona, journey, metrics, decisions, help
A file system	setup, brief, wiki, requirements, guide, styleguide, refine, figma (import, sync)
A shell and Node.js	prototype, storybook, package, showcase, skill-manager
Web search or browser tooling	competitor-analysis, ux-benchmark, showcase (screenshots)
A connected MCP server	figma (import, sync), workshop (FigJam board), map (FigJam export), requirements (tracker export)

When something is missing, a skill says so, returns its output in the conversation, and never claims a file was written when it was not.

In Claude Design, skills that need a shell and Node.js (`prototype`, `storybook`, `package`, `showcase`, `skill-manager`) will say so and stop instead of simulating output. Run those in Claude Code.

Audit

Audit finds problems and ranks them. It never edits your design. Name a mode, or it asks.

- **craft:** five dimensions scored 0 to 4 (total out of 20): visual hierarchy and cognitive load, information architecture, consistency, responsive behavior, delight. It also walks 2 to 3 personas through the main task. The output is a task list ranked P0 to P3.
- **heuristic:** a deeper pass against 12 usability heuristics, each issue rated 0 to 4 and tied to a screen. Use it when the audit has to stand as evidence.
- **a11y:** WCAG 2.1 and 2.2 AA, criterion by criterion.
- **code:** token use, structure, reuse and semantic HTML.
- **spec:** the build against the approved design, screen by screen, at every breakpoint.

Craft and heuristic scores are different instruments, so never average or merge them. Run modes one at a time. To fix findings, run `/refine`, then audit again to see whether the score moved. For a ship or handoff decision, ask for an independent second pass and reconcile both lists.

Plan

1. **Place the project.** Ask for the help skill. It reads which docs exist and recommends the next skill. An empty project gets `/setup`, then `/brief`.

2. **Follow the phases in order:** Discover, Decide, Design, Deliver. A project can skip ahead and backfill, for example by starting with a prototype and writing the research later.
3. **Plan before writing.** The repository's working rule is to ask questions one at a time, each with a recommendation, and wait for your confirmation before creating files. For anything bigger than a small tweak, say so and ask for the plan first.
4. **Do not redo finished work.** If a skill's document already exists and looks current, help will not recommend that skill again.

Guardrails

Nothing may be invented. These rules come from the repository's `AGENTS.md` and the skills themselves.

- **Unknowns before invention.** If a decision needs real user input, the answer goes on a line in `UNKNOWNS.md`, not into a guess. Answers move to the document that owns them once confirmed.
- **Confirmed beats invented.** Use something real: a file, a spec, your research. If nothing exists, a default is allowed only when it is labelled as a guess.
- **Name the source.** Every recommendation or finding points to the file, screen or your words that produced it.
- **No made-up people, quotes, numbers or competitors.** Interview quotes need an interview number. Competitors need a real source.
- **Stay in scope.** Audit never edits. Setup adds to existing files and does not overwrite them. Renaming an existing folder needs your explicit yes.
- **Never claim what did not happen.** A skill must not say a file was written, a command run or a tool called when it was not.
- **Safe settings.** Keep Claude on its default, which asks before editing. Do not switch on bypass or auto-accept. Never paste passwords, keys or tokens into a chat.

Paste this before any skill run to enforce them:

```
Use only files in this project and the skill's own SKILL.md. Do not invent users, quotes,
numbers, competitors or screens. If something is missing, add it to UNKNOWNS.md and ask m
e.
Show me a plan first and wait for my yes before creating or changing files. Name the sourc
e
file for every finding. If you guess, say it is a guess.
```

Avoiding slop

In text

- Praise words with nothing behind them: seamless, powerful, vibrant, pivotal. Say what happens instead.
- Big words for plain ones: leverage, utilize, facilitate, crucial. Use "use", "help", "important".
- Vague sources: "studies show", "experts believe". Name the source or cut the claim.
- Lists forced to three items, decorative em dashes, "it's worth noting", chatbot closers such as "I hope this helps".
- If you gave the text, it stays verbatim. Claude formats it and does not rewrite it.

In design

- No filler sections, placeholder copy or invented stats. Every element earns its place, and Claude asks before adding material.
- No aggressive gradients, emoji or stock-font defaults unless your brand uses them.
- Claude cannot generate photos. It says so and uses labelled placeholders.
- Small request, small change. If you ask for one edit, only that edit happens.

Why the skills do not appear when you type "/"

Typing `/` in the Claude Design input box shows no list of these skills, and you cannot add them to it from inside a project. They are plain files in your project, not features the app has registered. They work anyway because `CLAUDE.md` tells the assistant to read `skills/<name>/SKILL.md` when a message starts with `/<name>`. Type the full command and send it. `/help atlas` was tested this way and ran. If a command does nothing, check that Step 3 was done in this project and that `skills/<name>/SKILL.md` exists.

If something goes wrong

What you see	What to try
The assistant cannot see the repository	Install the GitHub App when prompted, and pick `RPalis/AIXLab-Atlas`.
`skills/` has more than 30 folders, or holds `competitors`, `scaffold`, `customer-journey`, `design-qa` or `user-flow`	That is an old copy. Delete `skills/` and repeat Step 2.
A command does nothing	Check Step 3 was done in this project and that `skills/<name>/SKILL.md` exists.
A skill writes before showing a plan	Type: stop, show me your plan first.
A skill says it cannot run a command or write a file	That is the Environment rule working. Take the output it returns in the conversation, or run that skill in Claude Code.